C++ COMPLETE REFERENCE LIBRARY • MODULE 07

07 ARRAYS AND STRINGS — Complete Theory & Study Book

Comprehensive Offline Theory, Memory Diagrams, Classroom C++ Programs, Practice Questions & Solutions

Module Table of Contents

1. **Module Overview & Study Guide** (README)
2. **Theory Chapter:** 01 1d and 2d arrays memory layout
3. **Theory Chapter:** 02 array decay and pointer equivalence
4. **Theory Chapter:** 03 c strings vs std string
5. **Visual Memory & Concept Diagrams** (1 Vector Diagram)
6. **Annotated C++ Source Programs** (4 Programs)
7. **Practice Question Bank:** conceptual questions
8. **Practice Question Bank:** debugging questions
9. **Practice Question Bank:** predict the output
10. **Practice Question Bank:** programming questions
11. **Step-by-Step Solutions:** conceptual solutions
12. **Step-by-Step Solutions:** output solutions
13. **Step-by-Step Solutions:** programming solutions

07 — Arrays and Strings

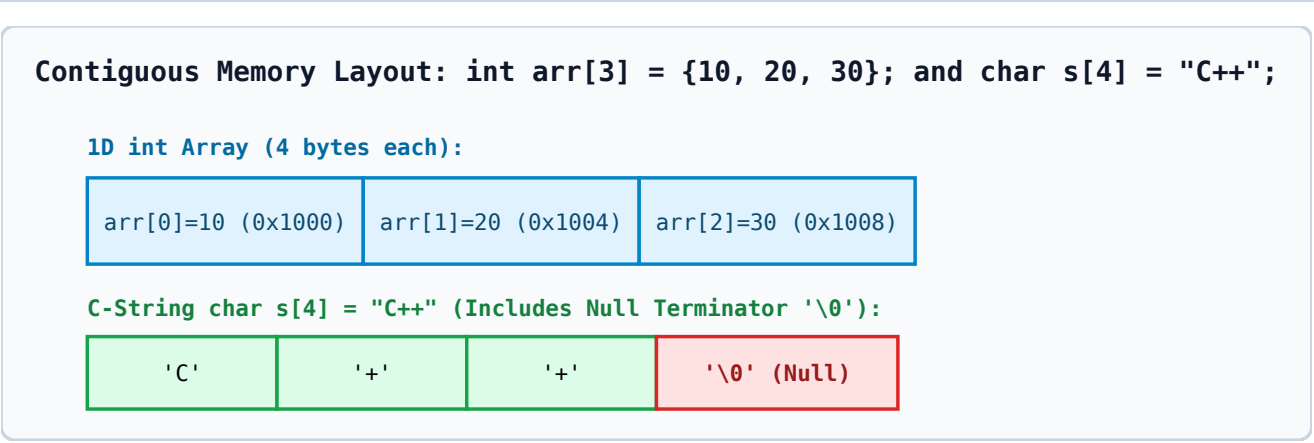
This module covers contiguous memory storage using 1D and 2D arrays, array-to-pointer decay rules, out-of-bounds Undefined Behavior, null-terminated C-style strings (`char[]`), and modern `std::string` .

Directory Contents

- **Theory (`01_THEORY/`)**:
  - `01_1d_and_2d_arrays_memory_layout.md` `(./01_THEORY/01_1d_and_2d_arrays_memory_layout.md)`
  - `02_array_decay_and_pointer_equivalence.md` `(./01_THEORY/02_array_decay_and_pointer_equivalence.md)`
  - `03_c_strings_vs_std_string.md` `(./01_THEORY/03_c_strings_vs_std_string.md)`
- **Programs (`02_PROGRAMS/`)**:
  - `01_array_traversal_and_sum.cpp` `(./02_PROGRAMS/01_array_traversal_and_sum.cpp)` (Category A)
  - `02_matrix_2d_addition.cpp` `(./02_PROGRAMS/02_matrix_2d_addition.cpp)` (Category A)
  - `03_c_string_vs_std_string.cpp` `(./02_PROGRAMS/03_c_string_vs_std_string.cpp)` (Category A)

  - `04_array_assignment_compile_error.cpp` (`./02_PROGRAMS/04_array_assignment_compile_error.cpp`) (Category B)
- **Practice Questions (`03_PRACTICE_QUESTIONS/`)** & **Detailed Solutions (`04_DETAILED_SOLUTIONS/`)**
- **Diagrams (`05_DIAGRAMS/`)**: `diagram_01_array_and_string_memory.svg` (`./05_DIAGRAMS/diagram_01_array_and_string_memory.svg`)

01 — 1D and 2D Arrays and Contiguous Row-Major Memory Layout



Array and String Memory Layout

1. Definition & Intuition

An **array** is a fixed-size, contiguous collection of elements of the exact same data type stored back-to-back in memory.

```cpp
int marks[5] = {80, 85, 90, 75, 95};
```

2. Address Calculation Formula

- **1D Array (`T arr[N]`)**:
  $$\text{Address}(\text{arr}[i]) = \text{BaseAddress}(\text{arr}) + i \times \text{sizeof}(T)$$
- **2D Array (`T mat[R][C]` , Row-Major Order in C++)**:
  C++ stores 2D arrays row by row in a single contiguous block: $$\text{Address}(\text{mat}[i][j]) = \text{BaseAddress}(\text{mat}) + (i \times C + j) \times \text{sizeof}(T)$$

3. Rules, Zero-Initialization, and Out-of-Bounds Undefined Behavior

1. **Compile-Time Constant Bound**: Standard C++ arrays (`int arr[N]`) require `N` to be a compile-time constant expression (`constexpr`). Variable Length Arrays (`int n; cin >> n; int arr[n];`) are a non-standard C99 compiler extension and are **ill-formed in strict ISO C++ (`-Wpedantic`)**.
2. **Partial Initialization**: `int arr[5] = {10, 20};` initializes `arr[0]=10, arr[1]=20` , and automatically zero-initializes `arr[2]=0, arr[3]=0, arr[4]=0` . Writing `int arr[5] = {};` zero-initializes all 5 elements.
3. **No Bounds Checking on `[]`** : Accessing `arr[5]` or `arr[-1]` on a 5-element array (`0..4`) is **Undefined Behavior**.

02 — Array-to-Pointer Decay and Subscript Equivalence

1. Subscript Identity (`arr[i] == *(arr + i)`)

By definition in the C++ standard (`[expr.sub]`), `arr[i]` is identical to `*(arr + i)`.

2. Why Arrays Cannot Be Assigned (`arr1 = arr2` Fails)

An array name `arr` is **not** a modifiable pointer variable; it is a non-modifiable lvalue designating the array object itself. Therefore:

- `arr1 = arr2;` -> **Compilation Error** (invalid array assignment).
- `arr++;` -> **Compilation Error** (lvalue required as increment operand).

3. Passing Arrays to Functions

When passed to a function `void print_array(int arr[], int n)`, the parameter `int arr[]` is adjusted by the compiler to `int* arr`. Consequently, `sizeof(arr)` inside the function returns `sizeof(int*)` (8 bytes), **not** the byte size of the caller's array! Always pass the element count `int n` alongside a raw array, or use `std::span<const int>` (C++20/23) or `std::array<int, N>`.

03 — C-Style Strings (`char[]`) vs. Modern `std::string`

1. C-Style Null-Terminated Character Arrays

A C-string is a `char` array terminated by the null character `'\0'` (ASCII `0`):

CPP

```cpp
char name[6] = "BTech"; // Stores {'B', 'T', 'e', 'c', 'h', '\0'} — requires 6 bytes for 5 letters!
```

If you write `char bad[5] = {'B','T','e','c','h'};` without `'\0'`, passing `bad` to `cout << bad` or `strlen(bad)` reads past the end of the array and causes **Undefined Behavior**.

2. Modern `std::string` (`#include <string>`)

CPP

```cpp
string course = "Mathematics";
course += " and Computing";
cout << course << " (Length: " << course.length() << ")" << endl;
```

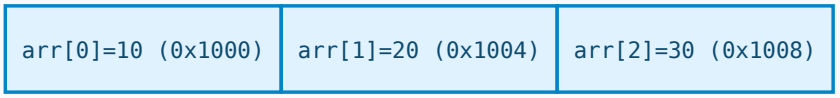
Why `std::string` is Preferred Over `char[]`

- Automatically resizes to prevent buffer overflows.
- Supports direct assignment (`s1 = s2`), concatenation (`s1 + s2`), and comparison (`s1 == s2`), whereas comparing two `char[]` arrays with `s1 == s2` compares their **memory addresses** instead of their text!

Visual Concept & Memory Diagrams (07 ARRAYS AND STRINGS)

Contiguous Memory Layout: int arr[3] = {10, 20, 30}; and char s[4] = "C++";

1D int Array (4 bytes each):



C-String char s[4] = "C++" (Includes Null Terminator '\0'):



Diagram: diagram 01 array and string memory

Annotated C++ Programs (07 ARRAYS AND STRINGS)

Program: `01_array_traversal_and_sum.cpp`

C++ SOURCE FILE • 01_array_traversal_and_sum.cpp

```cpp
// Program Title: 1D Array Traversal, Sum, and Maximum Element
// Topic: Arrays and Strings — 1D Arrays
// Purpose: Demonstrate declaring, initializing, and scanning a 1D integer array in classroom style.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/01_1d_and_2d_arrays_memory_layout.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

int main() {
    int marks[5] = {78, 92, 85, 64, 96};
    int sum = 0;
    int max_mark = marks[0];

    for (int i = 0; i < 5; i++) {
        sum = sum + marks[i];
        if (marks[i] > max_mark) {
            max_mark = marks[i];
        }
    }

    cout << "Total Sum: " << sum << endl;
    cout << "Highest Mark: " << max_mark << endl;

    return 0;
}

/*
Expected Deterministic Output:
Total Sum: 415
Highest Mark: 96
*/
```

Program: `02_matrix_2d_addition.cpp`

C++ SOURCE FILE • 02_matrix_2d_addition.cpp

```cpp
// Program Title: 2D Array Matrix Addition (2x2)
// Topic: Arrays and Strings — 2D Arrays
// Purpose: Demonstrate nested-loop matrix addition using 2D arrays in classroom style.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/01_1d_and_2d_arrays_memory_layout.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

int main() {
    int A[2][2] = {{1, 2}, {3, 4}};
    int B[2][2] = {{5, 6}, {7, 8}};
    int C[2][2];

    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            C[i][j] = A[i][j] + B[i][j];
        }
    }

    cout << "Result Matrix C:" << endl;
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            cout << C[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}

/*
Expected Deterministic Output:
Result Matrix C:
6 8
10 12
*/
```

Program: `03_c_string_vs_std_string.cpp`

C++ SOURCE FILE • 03_c_string_vs_std_string.cpp

```cpp
// Program Title: C-Style Null-Terminated String vs. Modern std::string
// Topic: Arrays and Strings — String Handling
// Purpose: Compare C-style char array traversal with modern std::string operations.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/03_c_strings_vs_std_string.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
#include <string>
using namespace std;

int main() {
    char c_str[6] = "Hello";
    string modern_str = "Hello";

    cout << "C-string: " << c_str << endl;
    modern_str = modern_str + " C++23";
    cout << "std::string: " << modern_str << " (Length: " << modern_str.length() << ")" <<
endl;

    return 0;
}

/*
Expected Deterministic Output:
C-string: Hello
std::string: Hello C++23 (Length: 11)
*/
```

Program: `04_array_assignment_compile_error.cpp`

C++ SOURCE FILE • 04_array_assignment_compile_error.cpp

```cpp
// INTENTIONAL COMPILE-TIME ERROR DEMONSTRATION
// Program Title: Direct Assignment and Increment of Array Names
// Topic: Arrays and Strings — Array Non-Modifiable Lvalue Rules
// Purpose: Show why assigning one raw array to another (a = b) or incrementing an array name
(a++) fails to compile.
// Target C++ Standard: C++23
// Related Theory File: ../01_THEORY/02_array_decay_and_pointer_equivalence.md
// Program Classification: Category B — Intentional compilation-error demonstration

#include <iostream>
using namespace std;

int main() {
    int a[3] = {1, 2, 3};
    int b[3] = {4, 5, 6};

    a = b; // COMPILE ERROR 1: invalid array assignment
    a++;   // COMPILE ERROR 2: lvalue required as increment operand

    return 0;
}

/*
Why Compilation Fails:
Raw C++ arrays cannot be assigned directly as whole objects, nor can the array identifier 'a'
be reseated.
To copy elements, either copy element-by-element in a loop, or use std::array<int, 3>
(#include <array>).
*/
```

Conceptual Questions — Module 07: Arrays and Strings

Q1. How is the memory address of element `arr[i][j]` computed in a 2D array `int arr[R][C]` stored in Row-Major order? **Q2.** Why does `char str[5] = "Hello";` fail to compile in C++ (whereas it was allowed in C)? **Q3.** Explain why `sizeof(arr)` inside `void func(int arr[])` returns the size of a pointer instead of the size of the array.

Debugging Questions — Module 07: Arrays and Strings

Q5. Explain the bug in comparing two C-strings using `if (s1 == s2)` when `char s1[] = "test"; char s2[] = "test";`, and show two ways to fix it.

Predict the Output Questions — Module 07: Arrays and Strings

Q4. Predict the output:

CPP

```cpp
#include <iostream>
using namespace std;
int main() {
    int a[5] = {10, 20};
    cout << a[0] << " " << a[1] << " " << a[2] << " " << a[4] << endl;
    return 0;
}
```

Programming Questions — Module 07: Arrays and Strings

Q6. Write a C++23 classroom program to compute the transpose of a 3×3 matrix and check whether the matrix is symmetric.

Detailed Solutions — Conceptual Questions (Module 07)

- **Q1 Solution**: $\text{Address}(\text{arr}[i][j]) = \text{Base} + (i \times C + j) \times \text{sizeof}(\text{int})$.
- **Q2 Solution**: The string literal `"Hello"` has 5 characters plus 1 mandatory null terminator `'\0'`, so its type is `const char[6]`. Initializing a 5-element `char[5]` from a 6-element string literal is ill-formed in C++.
- **Q3 Solution**: Function parameters declared as `int arr[]` are adjusted by the compiler to `int* arr` (array-to-pointer decay), so `sizeof(arr)` evaluates `sizeof(int*)`.

Detailed Solutions — Predict the Output & Debugging (Module 07)

- **Q4 Solution**: Partial brace initialization `{10, 20}` sets `a[0]=10, a[1]=20` and zero-initializes all remaining elements (`a[2]=0, a[3]=0, a[4]=0`). Output: `10 20 0 0` .
- **Q5 Solution**: `s1 == s2` decays both arrays to pointers (`char*`) and compares their distinct stack addresses (`false`) instead of their characters! Fix 1: Use `std::string s1 = "test", s2 = "test"; if (s1 == s2)` . Fix 2: Use `strcmp(s1, s2) == 0` (`#include <cstring>`).

Detailed Solutions — Programming Questions (Module 07)

Solution to Q6 (Matrix Transpose & Symmetry Check)

CPP

```cpp
#include <iostream>
using namespace std;

int main() {
    int m[3][3] = {{1, 2, 3}, {2, 5, 6}, {3, 6, 9}};
    bool symmetric = true;

    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (m[i][j] != m[j][i]) {
                symmetric = false;
            }
        }
    }

    cout << "Is symmetric? " << symmetric << endl;
    return 0;
}
// Expected Output: Is symmetric? 1
```